



DD2395 Computer Security

DD2391 Cybersecurity Overview

Re-exam 2023-12-20 08:00–12:00

The problems are *not* sorted according to any perceived difficulty.

Students are allowed to bring any course materials in physical form including slides, course books, past exams, and own notes.

The exam grade is computed as follows:

- FX: 30-34 points
- E: 35-44 points
- D: 45-54 points
- C: 55-64 points
- B: 65-74 points
- A: 75-100 points

- 1 Authentication - Software Integrity [14 points]** Alice is the owner of company Awesome, which operates in the financial sector. She uses the software product Magic of software company TamperProof to manage the finances of her own company. Recently, Alice has found out that the executable binary of Magic, `bin.exe`, which she received from TamperProof (along with other cryptographic information), is malicious and has been stealing sensitive files from her computer. Alice is considering of taking TamperProof to court, but she is not certain she has sufficient evidence to prove that the binary `bin.exe` is indeed from TamperProof. Alice asks for your advice and presents as evidence the malicious binary `bin.exe`, along with some cryptography applied to it.

Your task is to decide whether each piece of cryptographic evidence below is sufficient to prove Alice's claim that the malicious binary `bin.exe` is from TamperProof. You can assume that all of the subproblems below are independent of each other. Each subproblem is worth 3 points and you only receive 1 point if you provide the correct answer (Yes/No) with no justification.

The exercise is subject to the following technical details and assumptions:

- `pk` and `sk` denote a public key and a private key, respectively.
- `MAC` is a cryptographically secure message authentication code function.
- `k1` and `k2` both denote secret keys shared between Alice and TamperProof.
- `H` is a cryptographically secure hash function.

- $\text{Sign}(\text{sk}, \text{msg})$ is a digital signature algorithm signing a message msg with secret key sk . As expected, you can verify any signature you are presented with against the corresponding public key.
- Enc and Dec denote an IND-CPA secure symmetric encryption scheme.

1a [3 points] In addition to `bin.exe`, Alice presents as evidence $\text{Sign}(\text{sk}, \text{bin.exe})$ and pk . You can obtain $\text{Sign}(\text{sk1}, \text{"TamperProof's public key is pk"})$ from a trusted certificate authority, i.e., a certificate authority trusted by everyone, including you, TamperProof and the court. sk1 is the secret key of the certificate authority, and you know the corresponding public key pk1 . Justify whether or not this piece of evidence proves that `bin.exe` is from TamperProof.

Solution Since the certificate authority is trusted, you trust that the public key pk actually belongs to TamperProof. Then you can use TamperProof's public key to verify that the signature is valid. This proves that `bin.exe` is from TamperProof.

1b [3 points] In addition to `bin.exe`, Alice presents as evidence $H(\text{bin.exe})$. Justify whether or not this piece of evidence proves that `bin.exe` is from TamperProof.

Solution This cannot serve as evidence because Alice can compute the hash on any message by herself.

1c [2 points] In addition to `bin.exe`, Alice presents you with $\text{MAC}(\text{k1}, \text{bin.exe})$ and the secret key k1 . Justify whether or not this piece of evidence proves that `bin.exe` is from TamperProof.

Solution This cannot serve as evidence because Alice can compute the MAC of any binary since she has the secret keys k1 .

1d [3 points] In addition to `bin.exe`, Alice presents you with $\text{MAC}(\text{k1}, \text{Enc}(\text{k2}, \text{bin.exe}))$ and the secret keys k1 and k2 . Justify whether or not this piece of evidence proves that `bin.exe` is from TamperProof.

Solution This cannot serve as evidence because Alice can compute the MAC and the encryption since she has the secret keys k1 and k2 .

1e [3 points] In addition to `bin.exe`, Alice presents as evidence $\text{Sign}(\text{sk}, \text{bin.exe})$ and pk . Moreover, Alice generates a certificate with her private key $\text{Sign}(\text{sk2}, \text{"TamperProof's public key is pk"})$ and presents as evidence the certificate and her public key pk2 (note that sk2 is the private key of Alice). Justify whether or not this piece of evidence proves that `bin.exe` is from TamperProof.

Solution This does not prove that `bin.exe` is from TamperProof, because Alice can be lying about TamperProof's public key pk .

2 Multi Level Security [12 points]

Consider the Bell-LaPadula (BLP) multi-level security model. Recall that in the BLP model, labels L are pairs (SL, C) of sensitivity levels SL and compartments (sets of areas) C , and they are used to enforce the "no read up" and "no write down" rules. Consider a company operates on documents of two levels of sensitivity, Secret (S), and Public (P) such that $P \sqsubseteq S$, in the areas of Web (W), Network (N), and Forensics (F).

- 2a [6 points]** Provide the definition of "no read up" and "no write down" rules (2 points in total) for a subject S with label $L_S = (SL_S, C_S)$ and an object O with label $L_O = (SL_O, C_O)$. Write out (i.e., draw) the label partial order that meets requirements of the company (4 points in total).

Solution No read up: S may read O iff $L_O \sqsubseteq L_S$. No write down: S may write O iff $L_S \sqsubseteq L_O$. A drawing of the subset partial order such that for any $L1 = (SL1, C1)$ and $L2 = (SL2, C2)$, we have that $L1 \sqsubseteq L2$ iff $SL1 \sqsubseteq SL2$ and $C1 \subseteq C2$, where Ci is a subset of $\{W, N, F\}$.

- 2b [6 points]** Consider a subject S with label $L_S = (S, \{W, F\})$, an object A with label $L_A = (S, \{W, F\})$, and an object B with label $L_B = (P, \{W\})$.

- [3 points]** Use the rules of the BLP model to show whether or not the subject S can read document A and write it to document B .
- [3 points]** The company now considers the BLP model too restrictive for its operations and decides to relax the policy by removing the no write down rule and keeping only the no read up rule. Use the rules of this relaxed BLP model to show whether or not the subject S can read document A and write it to document B .

Solution For the first point, it is not possible as shown in slide 81 of Lecture 3. For the second point, it is now possible since the no read up rule ($L_O \sqsubseteq L_S$) does not prevent S from reading A and writing it to B .

3 Web Security [12 points] The website `https://champagne.com` handles online purchase of expensive champagne brands for registered users. Recently, they have been subject to Cross Site Request Forgery (CSRF) attacks, hence they have decided to defend against these attacks by implementing the following countermeasures:

- When a user logs in, `champagne.com` sets two 64-byte cookies, `sessionId` and `csrfToken`, randomly with domain `champagne.com`.
- When a user sends a POST request, the value of the `csrfToken` is embedded as one of the form fields. On receiving the POST request, `champagne.com` checks that the value of `csrfToken` cookie matches the one in the form.

You can assume that the cookies do not have the Secure, HTTPOnly, or Strict flags set unless stated otherwise. You can also assume that no CSRF defenses besides the tokens are implemented. Finally, you can assume every subproblem is independent, i.e, the solution is independent of the previous subproblems.

Your task is to answer and justify, for every subproblem, each of three statements below:

1. The attacker can carry out a CSRF attack against `https://champagne.com`
2. The attacker can change the user's `csrfToken` cookie
3. The attacker can learn the value of the user's `sessionId` cookie

For each subproblem, you can earn a maximum of 3 points (1 point for correct answer and justification of each statement). You will only receive 1 point if all of the answers (Yes/No) are correct, but the justification (of at least one statement) is missing or is incorrect.

3a [3 points] Suppose the attacker gets the user to visit their malicious website which has domain `https://attacker.com`. Justify the validity of each of the three statements above.

Solution The attacker's website is on a different domain so they are not able to either change or read any cookies for `https://champagne.com`. As a result, they are not able to execute a CSRF attack since they cannot guess the value of `csrfToken`. Hence the answer is No for all statements.

3b [3 points] Suppose the attacker gets the user to visit their malicious website which has domain `https://attacker.champagne.com`. Justify the validity of each of the three statements above.

Solution Since the attacker's website `https://attacker.champagne.com` is a subdomain of `https://champagne.com`, it can both read and set cookies. The attacker can embed JavaScript in their page to extract `csrfToken` and form a malicious POST request. Hence the answer is Yes for all statements.

3c [3 points] Suppose the attacker gets the user to visit a page on the website `https://storedxss.champagne.com` that contains a stored XSS vulnerability, and the website `https://storedxss.champagne.com` is not controlled by the attacker. Justify the validity of each of the three statements above.

Solution The attacker can use the stored XSS vulnerability to extract the `csrfToken` cookie and cause the user's browser to make a malicious POST request. Similarly to the previous subproblem, the attacker can read and set the cookies. Hence, the answer is Yes for all statements.

- 3d [3 points]** Suppose the `csrfToken` and `sessionId` cookies have the HTTPOnly flag set. Suppose the attacker gets the user to visit a page on the website `https://storedxss.champagne.com` that contains a stored XSS vulnerability, and the website `https://storedxss.champagne.com` is not controlled by the attacker. Justify the validity of each of the three statements above.

Solution The HTTPOnly flag renders the XSS attack useless for a CSRF attack since JavaScript cannot extract the value of `csrfToken` and `sessionId`. Hence, the answer is No for all statements.

4 Hash functions [12 points]

Let $x = x_1 \dots x_n$ be an input of n blocks of 256 bits. Let $H(x) = SHA(x_1) \oplus x_2 \oplus \dots \oplus x_n$, where $\oplus$ is xor and SHA the Secure Hash Algorithm 2.

- 4a [5 points]** Instantiate each requirement of a cryptographically secure hash functions to the definition of H . This means that you should give the definition of each requirement of a cryptographically secure hash function in terms of the definition of H .
- 4b [7 points]** For each requirement, disprove the requirement or argue why the requirement is met.

Solution

- Weak collision resistance: given $x_1 \dots x_n$ it should be infeasible to find a different $y_1 \dots y_n$ such that $SHA(x_1) \oplus x_2 \oplus \dots \oplus x_n = SHA(y_1) \oplus y_2 \oplus \dots \oplus y_n$. It is not weak collision resistant, since for $n > 1$ blocks, every permutation of the blocks produces the same hash value.
- Strong collision resistant: it should be infeasible to find any $x_1 \dots x_n$ and $y_1 \dots y_n$ such that $SHA(x_1) \oplus x_2 \oplus \dots \oplus x_n = SHA(y_1) \oplus y_2 \oplus \dots \oplus y_n$. It is not weak collision resistant, therefore it is not strong collision resistance.
- One way: given h it should be infeasible to find $x_1 \dots x_n$ such that $SHA(x_1) \oplus x_2 \oplus \dots \oplus x_n = h$. It is not one way. For example $H(aah) = h$ (where a is an arbitrary block), since $SHA(a) \oplus SHA(s) \oplus h = h$.

5 Protocol [12 points]

In a simple doorbell protocol, a remote control C (doorbell) and a receiver R (authenticator) communicate over a reliable cable. They share a secret K . The function H is a cryptographically secure hash function.

1. C sends $c = H(K||t)$, where t is the current time in the format HH:MM:SS (hours, minutes and seconds).
2. R verifies if $c == H(K||t')$ for t' their current time HH:MM:xx where xx are any of the values from 00 through 59, then R initiates opening of a door over a channel that guarantees authenticity and integrity

5a [4 points] Describe a DoS attack and state the assumptions on the attacker capabilities. Who is the target and what is the outcome of the attack? **[2 points]** Suggest a change to the protocol that mitigates the problem. For this mitigation describe a limitation or why there are none.

Solution Replay of intercepted messages can cause resource exhaustion at R, as 60 comparisons happen. Suggestions: disregard seconds and compare only the full timestamp rounded to the minute (only 1 comparisons), include the day as well. An attacker that intercepts this message anyways can replay such message however at a higher cost for the attacker.

5b [3 points] Transform the messages in the protocol such that authenticity and integrity is guaranteed. Carefully describe which changes guarantee authenticity and which changes ensure integrity. Include any further assumptions that are required.

Solution MAC-then-encrypt the timestamp instead of hashing. MAC ensures authenticity (assuming that only two people have the symmetric key; *authentication by encryption*) and integrity.

5c [3 points] Transform the messages in the protocol such that confidentiality is guaranteed. Include any further assumptions that are required.

Solution Asymmetric encryption ensures confidentiality, and assumes that the public key of R is known to C. Alternatively symmetric encryption with a shared private key will ensure confidentiality.

6 Memory safety. [14 points] The following program is not memory safe:

```
1 void a(void) { ... }
2 void b(void) { ... }
3 void (*fun_ptr)(void) = &a;

4 void check_pin() {
5     int * x = malloc(sizeof(int));
6     char buffer[10];
7     FILE * f = fopen("input.txt", "r");
8     fscanf(f, "%s", buffer);
9     fscanf(f, "%d", x);
10    fun_ptr();
11    free(x);
12    fclose(f);
13    return;
14 }
```

Here `f = fopen("input.txt", "r");` open an input file, `fscanf(f, "%s", buffer);` reads from a file a string until a whitespace is found and copies the string in `buffer`, `fscanf(f, "%d", x);` reads from a file a string, parses the string as integer, and copies the integer to the memory location pointed by `x`. Also, `void (*fun_ptr)(void) = &a` declares a function pointer that points to function `a` and `fun_ptr()` invokes the function pointed by `fun_ptr`. Assume that the attacker controls the file read by the program.

- 6a [2 points]** Identify the lines of the program that have a problem and explain why they are not secure.
- 6b [3 points]** Explain a mechanism that can be used by the attacker to force the program to invoke function `b` **after** line 11. In your answer include a description of what should be in the file read by the program. **[2 points]** Explain if (and why/why not) this attack can be countered by stack canaries.
- 6c [3 points]** Explain a mechanism that can be used by the attacker to force the program to invoke function `b` **before** line 11. In your answer include a description of what should be in the file read by the program. **[2 points]** Explain if (and why/why not) this attack can be countered by stack canaries.
- 6d [2 points]** If one of the two attacks in 6.b or 6.c cannot be countered by stack canaries, propose and motivate a countermeasure.

Solution

- `fread` at line 8 can overflow `buffer` since it can write more than 10 bytes if the file has more than 10 consecutive characters without a whitespace (or newline).

- assume that on the stack we have (from low, to high addresses) `buffer`, `x`, `return pointer`. We assume the architecture is 64 bits (i.e., pointers are 8 bytes) If the file contains `123456789012345678<address of b>`, the `fread` at line 8 overflows the stack variables and overwrite the return pointer with the address of `b`. Therefore, when `check_pin` returns it jumps to `b`. This bug can be countered by a stack canary, which is placed between `x` and the return pointer. Assuming that the attacker does not know the value of a random canary, the file will contain `123456789012345678<new value for the canary><address of b>` and the buffer overflow will overwrite the value of the canary. When the function returns, it can check that the canary has been altered, detecting the overflow on the stack.
- assume that on the stack we have (from low, to high addresses) `buffer`, `x`, `return pointer`. We assume the architecture is 64 bits (i.e., pointers are 8 bytes) If the file contains `1234567890<address of fun_ptr> <address of b>`(notice the space between the two addresses), the `fread` at line 8 overflows the stack variables and change `x` to point to `fun_ptr`. The next `fread` at line 9 reads the integer after the space, however since `x` to point to `fun_ptr` it writes the integer into `fun_ptr`, setting the function pointer to point to `b`. Therefore, when `check_pin` calls `fun_ptr()` at line 10 it jumps to `b`. This bug cannot be countered by a stack canary, since it will be usually placed after `x` and therefore it will not be affected by this overflow.
- A simple countermeasure to this attack is ASLR, since it prevents the attacker to know the address of `b`.

- 7 Side channels. [12 points]** Suppose that a system uses a shared data cache (no instruction cache) of 256 bytes, 4 lines, 64 bytes per line. The cache is directly mapped. The array M consists of 32 entries of 4 bytes. The first sixteen entries of the array are mapped to the first cache line, the second sixteen entries are mapped to the second cache line. The array is public and contains $M[x] = 2 * ((x \bmod 16) + 1)$ (i.e. $M[0]=2$; $M[1]=4$; $M[2]=6 \dots M[15]=32$; $M[16]=2 \dots M[31]=32$). The variables A and B are secret numbers and C and M are a public variables that are known by the attacker. Variables A , B and C are all allocated in the same block of 64 bytes, which is mapped on the third cache line. Finally, assume that A , B , and C are smaller than 32. The victim program is the following:

```

1 WHILE (A > 0)
2     A -= M[C]
3 A += M[B]

```

Suppose that the attacker can **only** measure the execution time of the victim program, but it is allowed to observe multiple executions with the same initial values for A , B , M , and all possible values of C . By using multiple measurements, the attacker can extract some information about the secret variables of this program.

- 7a [3 points]** Explain why this program is not secure and which instructions can cause information leakage.

Solution Iterations of loops (1) depend on the initial value of A . Instruction (3) accesses an address that depends on the the value of B

- 7b [5 points]** Explain why information is leaked and how the attacker can correlate different executions to learn different information of the initial values of the variables.

Solution First, we focus on the case where $C \in \{0 \dots 15\}$. In all these case, line (2) always fetches the same block of M into the cache, so after the first time, all other accesses result in a cache hit. «««< HEAD For all these executions, if $B > 16$ then line (3) always results in a cache miss, otherwise it always results in a cache hit. Therefore the execution times will depend on the number of iteration of the loop, which is equal to $A/(2 * C)$. In particular, all executions for $C > A/2$ lead to only one execution of the loop and therefore have the same execution time. The attacker can simply ===== For all these executions, if $B > 16$, then line (3) will always results in a cache miss, otherwise it always results in a cache hit. Therefore the execution times will depend on the number of iteration of the loop, which is equal to $A/2 * C + 1$. In particular, all executions for $C > A/2$ will lead to only one execution of the loop and therefore will have the same execution time. The attacker can simply >>>>> 21e8a9cefd7e1ceed6bd6bd0f9dfbb007fe2433c select the smallest C_{min} with the largest execution time to find out the that A is either $2 * C_{min}$ or $2 * C_{min} - 1$.

We not look at two executions: $C_1 = 0$ and $C_2 = 16$. In both cases $M[C_1] = M[C_2] = 2$ so the number of loop iterations are the same. However, in the first case, line (2) has fetched the first 16 elements of M in the cache, in the second case it has fetched the second 16 elements of M . The different execution time for C_1 and C_2 therefore only depends on line (3). If $B < 16$ then the execution for C_1 is faster then the case for C_2 , since line (3) results in a cache hit for C_1 and in a cache miss for C_2 . If $B \geq 16$ then the execution for C_1 is slower then the case for C_2 , since line (3) results in a cache miss for C_1 and in a cache hit for C_2 .

7c [4 points] Describe how you can fix the program or prevent the attack.

Solution The simplest solution is first to guarantee that the execution of the loop is done the same amount of times. This can be done using conditional assignment (using additional variables that we assume to now cause eviction of other variables and assuming that conditional assignment can be done in constant time).

```
1 FOR (I = 0 to 16) // Worst case scenario
2   newA = A-M[C]
3   A = (A>0)?newA:A
```

Of course we can also remove the loop

$A = -M[C] + (A \bmod M[C])$

Then we can prime the cache accessing both $M[0]$ and $M[16]$. This would guarantee that we always access lines $[0,1]$ and therefore the old line (3) will always results in a cache hit.

8 Block cipher mode of operation. [12 points]

We discuss a novel mode of operation to calculate for given plaintext $m_1 || \dots || m_n$ of n blocks the ciphertext $c_1 || \dots || c_n$, where $||$ is concatenation. AES encryption for a key K and plaintext M is written $E_K(M)$, and $\oplus$ is bit-wise addition.

$$\begin{aligned}c_0 &= 1010 \dots 01 \quad (\text{a constant vector}) \\c_1 &= E_K(K) \oplus c_0 \oplus m_1 \\c_2 &= E_K(K) \oplus c_0 \oplus m_2 \\c_i &= E_K(c_{i-2}) \oplus E_K(c_{i-1}) \oplus m_i \quad (\text{for } i > 2)\end{aligned}$$

8a [2 points] State the decryption formulae that invert the block encryption. Use D_K for AES decryption, if needed.

Solution 1 point for including $m_1 = E_K(K) \oplus c_0 \oplus c_1$ and $m_2 = E_K(K) \oplus c_0 \oplus c_2$, 1 point for correctly decoding $m_i = c_i \oplus E_K(c_{i-2}) \oplus E_K(c_{i-1})$ for $i > 2$.

8b [4 points] Explain whether or not the encryption is IND-CPA secure and otherwise discuss a possible fix.

Solution The encryption is deterministic (1 point), thus not IND-CPA secure due to a constant initialisation vector c_0 (IV) (1 point). With random IV (1 point) the mode becomes IND-CPA secure as the non-determinism of IV propagates through all of c_i (another point for the argument).

If an error in transmission of block c_i only causes finitely many blocks $m_i \dots m_{i+n}$ to be wrongly decrypted then the mode is called *self-correcting*.

8c [6 points] Argue whether or not this mode is self-correcting and support your reasoning. Assume an error in the transmission of block c_4 . Which blocks will be decrypted correctly and which blocks be decrypted wrongly (if applicable)?

Solution The mode is self-correcting: Block c_4 is needed to correctly decrypt the three blocks $m_4 = E_K(c_2) \oplus E_K(c_3) \oplus c_4$, $m_5 = E_K(c_3) \oplus E_K(c_4) \oplus c_5$ and $m_6 = E_K(c_4) \oplus E_K(c_5) \oplus c_6$. (3 points) The error does not propagate to m_i for $i > 7$ (2 points), neither to m_1, m_2, m_3 . (1 point)